

Experiment 4.

Student Name: Sparsh Mehta

Branch: BE-IT

Semester: 7th

Subject Name: Competitive Coding-III

UID: 23BET10092

Section/Group: 715-‘B’

Date of Performance: 13th Aug, 2026

Subject Code: 23ITP-403

1.) Aim: To Analysis of the Longest Substring Without Repeating Characters Using the Sliding Window Optimization Technique.

2.) Objective:

- To understand the problem of finding the longest substring containing unique characters.
- To implement the Sliding Window optimization technique for efficient string processing.
- To analyze and compare the efficiency of the optimized approach with a brute-force approach.
- To apply data structures such as HashSet/HashMap for tracking characters within the current window.
- To analyze the time and space complexity of the implemented algorithm.

3.) Problem Statement: (A.) Fixed size Sliding Window 643. Maximum Average Subarray I, You are given an integer array nums consisting of n elements, and an integer k. Find a contiguous subarray whose length is equal to k that has the maximum average value and return this value. Any answer with a calculation error less than 10⁻⁵ will be accepted.

Example 1: Input: nums = [1,12,-5,-6,50,3], k = 4Output: 12.75000

Link: <https://leetcode.com/problems/maximum-average-subarray-i/description/>

(B.) Variable size Sliding Window 424. Longest Repeating Character Replacement, You are given a string s and an integer k. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times. Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1: Input: s = "ABAB", k = 2

Output: 4

Link: <https://leetcode.com/problems/longest-repeating-character-replacement/description/>

(C.) Advanced Sliding Window 992. Subarrays with K Different Integers, Given an integer array nums and an integer k, return the number of good subarrays of nums.

Example 1: Input: nums = [1,2,1,2,3], k = 2

Output: 7

Link: <https://leetcode.com/problems/subarrays-with-k-different-integers/description/>

4.) Code:

(A.)

```
class Solution {
    public double findMaxAverage(int[] nums, int k) {
        //NAME: SPARSH MEHTA
        //UID: 23BET10092
        int sum = 0;
        for (int i = 0; i < k; i++) {
            sum += nums[i];
        }
        int maxSum = sum;
        int startIndex = 0;
        int endIndex = k;
        while (endIndex < nums.length) {
            sum -= nums[startIndex];
            startIndex++;
            sum += nums[endIndex];
            endIndex++;
            maxSum = Math.max(maxSum, sum);
        }
        return (double) maxSum / k;
    }
}
```

(B.)

```
class Solution {
    public int characterReplacement(String s, int k) {
        //NAME: SPARSH MEHTA
        //UID: 23BET10092
        int [] freq = new int[26];
        int left = 0;
        int maxFreq = 0;
        int maxWindow = 0;
        for(int right = 0; right < s.length(); right++){

            freq[s.charAt(right)-'A']++;
            maxFreq = Math.max(maxFreq, freq[s.charAt(right)-'A']);
```

Discover. Learn. Empower.

```
int windowLength = right - left + 1;  
if(windowLength - maxFreq > k){
```

```
    freq[s.charAt(left)-'A']--;  
    left++;  
}  
windowLength = right - left + 1;  
maxWindow = Math.max(maxWindow, windowLength);  
}  
return maxWindow;  
}  
}
```

(C.)

```
class Solution {  
    public int subarraysWithKDistinct(int[] nums, int k) {  
        //NAME: SPARSH MEHTA  
        //UID: 23BET10092  
        return atMost(nums, k) - atMost(nums, k - 1);  
    }  
    private int atMost(int[] nums, int k) {  
        int[] freq = new int[nums.length + 1];  
        int left = 0;  
        int distinct = 0;  
        int result = 0;  
        for (int right = 0; right < nums.length; right++) {  
            if (freq[nums[right]] == 0) {  
                distinct++;  
            }  
            freq[nums[right]]++;  
            while (distinct > k) {  
                freq[nums[left]]--;  
                if (freq[nums[left]] == 0) {  
                    distinct--;  
                }  
                left++;  
            }  
            result += right - left + 1;  
        }  
        return result;  
    }  
}
```

(A.)

</> Code

Java ▾ 🔒 Auto

```

1 class Solution {
2     public double findMaxAverage(int[] nums, int k) {
3         //NAME: SPARSH MEHTA
4         //UID: 23BET10092
5         int sum = 0;
6         for (int i = 0; i < k; i++) {
7             sum += nums[i];
8         }
9         int maxSum = sum;
10        int startIndex = 0;
11        int endIndex = k;
12        while (endIndex < nums.length) {
13            sum -= nums[startIndex];
14            startIndex++;
15            sum += nums[endIndex];
16            endIndex++;
17            maxSum = Math.max(maxSum, sum);
18        }
19        return (double) maxSum / k;
20    }
21 }
```

(B.)

</> Code

Java ▾ 🔒 Auto

```

1 class Solution {
2     public int characterReplacement(String s, int k) {
3         //NAME: SPARSH MEHTA
4         //UID: 23BET10092
5         int [] freq = new int[26];
6         int left = 0;
7         int maxFreq = 0;
8         int maxWindow = 0;
9         for(int right = 0; right < s.length(); right++){
10             freq[s.charAt(right) - 'A']++;
11             maxFreq = Math.max(maxFreq, freq[s.charAt(right) - 'A']);
12             int windowLength = right - left + 1;
13             if(windowLength - maxFreq > k){
14                 freq[s.charAt(left) - 'A']--;
15                 left++;
16             }
17             windowLength = right - left + 1;
18             maxWindow = Math.max(maxWindow, windowLength);
19         }
20         return maxWindow;
21     }
22 }
```

(C.)

</> Code

Java ▾ 🔒 Auto

```

1 class Solution {
2     public int subarraysWithKDistinct(int[] nums, int k) {
3         //NAME: SPARSH MEHTA
4         //UID: 23BET10092
5         return atMost(nums, k) - atMost(nums, k - 1);
6     }
7     private int atMost(int[] nums, int k) {
8         int[] freq = new int[nums.length + 1];
9         int left = 0;
10        int distinct = 0;
11        int result = 0;
12        for (int right = 0; right < nums.length; right++) {
13            if (freq[nums[right]] == 0) {
14                distinct++;
15            }
16            freq[nums[right]]++;
17            while (distinct > k) {
18                freq[nums[left]]--;
19                if (freq[nums[left]] == 0) {
20                    distinct--;
21                }
22                left++;
23            }
24            result += right - left + 1;
25        }
26        return result;
27    }
28 }
```



5.) Output:

(A.)

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

```
nums =  
[1, 12, -5, -6, 50, 3]
```

```
k =  
4
```

Output

```
12.75000
```

Expected

```
12.75000
```

(C.)

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

```
nums =  
[1, 2, 1, 2, 3]
```

```
k =  
2
```

Output

```
7
```

Expected

```
7
```

(B.)

☒ Testcase | [> Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

```
s =  
"ABAB"
```

```
k =  
2
```

Output

```
4
```

Expected

```
4
```

6.) Learning Outcomes:

- Implement an efficient solution for finding the longest substring without repeating characters.
- Apply the Sliding Window technique to solve substring and subarray problems.
- Use HashSet or HashMap to efficiently detect and manage duplicate characters.
- Optimize a brute-force string solution from $O(n^2)$ to $O(n)$ time complexity.
- Analyze algorithm performance in terms of time complexity, space complexity, and scalability.